

Code-Layout, Readability and Reusability

Date	30 June 2026
Team ID	Not Provided
Project Name	Smart Lender
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the Smart Lender codebase in terms of layout, naming, readability, and reuse. The project is organized as a compact Flask and machine-learning application with a clear separation between training logic, runtime prediction logic, templates, static assets, model artifacts, and tests. The existing code is sufficiently readable for academic submission and practical maintenance.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Python source files use consistent indentation and readable section separators.
2	Proper File Structure	Files and folders are logically organized	Yes	The repository separates app.py, train_model.py, templates, static, model, data, and tests clearly.

3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names such as <code>feature_columns</code> , <code>model_comparison</code> , <code>loan_amount_term</code> , and <code>credit_history</code> are descriptive.
4	Function / Method Names	Functions are descriptively named	Yes	Functions like <code>predict_loan_status</code> , <code>predict_batch</code> , <code>clean_and_engineer</code> , and <code>_get_session_stats</code> clearly communicate intent.
5	Code Comments	Inline and block comments explain logic	Yes	The code includes module docstrings, route summaries, and short explanatory comments in complex sections.
6	Modular Design	Code is split into reusable functions/modules	Yes	Preprocessing, inference, batch processing, reporting, and training are broken into reusable helpers and separate modules.
7	No Redundant Code	Duplicate or unused code is removed	Partial	Some configuration constants and feature label definitions appear in both <code>app.py</code> and <code>train_model.py</code> , but the duplication is limited and understandable.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	The application handles missing model artifacts, invalid form data, invalid API payloads, and unsupported file uploads without crashing.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology Where Reused	Reusability Level (High / Medium / Low)
1	<code>build_feature_row()</code>	Python preprocessing reused by HTML form prediction and JSON API prediction	High
2	<code>predict_loan_status()</code>	Python inference logic reused by <code>/predict</code> and <code>/api/predict</code>	High
3	<code>predict_batch()</code>	Python batch scoring reused by CSV upload workflow and downloadable result generation	High
4	<code>_calc_emi()</code>	Python business logic reused by result-page calculations and <code>/api/emi</code>	High
5	<code>clean_and_engineer()</code>	Python model-preparation routine reused during training, evaluation, and regression tests	High

Overall Code Quality Assessment:

A spect	Rating (1-5)	Comments
Code Layout & Structure	4	The repository is well partitioned for a small Flask ML project and is easy to navigate.
Readability	4	Clear naming, section headers, and route organization make the code understandable with minimal ramp-up time.
Reusability	4	Core logic is encapsulated in reusable helpers rather than duplicated across routes.
Documentation / Comments	4	README instructions, docstrings, and inline explanations are present where they add value.

Overall Score	4	The codebase is maintainable, testable, and suitable for extension, with only minor opportunities for further DRY refactoring.
---------------	---	--